

IBM DATA SCIENCE CAPSTONE

SpaceX Falcon 9

First Stage Landing Prediction

Predicting rocket reusability using machine learning to estimate launch costs and compete with SpaceX

Temisan Arubi, Space Y Analytics Team · 2026

Presentation Outline

01 Title Slide
SpaceX Falcon 9 — First Stage Landing Prediction

02 Presentation Outline
All 18 slides at a glance

03 Executive Summary
Business problem, approach & key outcome

04 Introduction
Commercial space industry & project context

05 Data Collection & Wrangling Methodology
API, web scraping, ID resolution, null handling, class labeling

06 EDA & Visual Analytics Methodology
Visualization, SQL, Folium maps, Plotly Dash

07 Predictive Analysis Methodology
Pipeline, preprocessing, 4 classifiers, GridSearchCV

08 Data Collection — SpaceX API
REST API, GET /v4/launches/past, ID resolution, dataset_part_1.csv

09 Data Collection — Web Scraping
BeautifulSoup, Wikipedia HTML tables, 10 columns parsed

10 Data Wrangling
Null handling, outcome classification, Class 0/1 label, dataset_part_2.csv

11 EDA with Visualization Results
6 charts: flight no., payload, site, orbit, trend

12 EDA with SQL Results
10 SQL tasks: sites, orbits, payloads, outcomes, ranked analysis

13 Interactive Map Results — Folium
Site markers, success/fail colours, proximity distances

14 Plotly Dash Dashboard Results
Dropdown, slider, pie chart, scatter — real-time interactivity

15 Predictive Analysis Results
Decision Tree 88.9%, vs LR/SVM/KNN at 83.3%

16 Conclusion
90 records, KSC 55% success, 88.9% accuracy, \$62M cost insight

17 Innovative Insights
Coastal geography, payload sweet spot, 2015 pivot, non-linear ML

18 GitHub Repository
All notebooks, Dash app — github.com/Temiarubi/spacex-project

Executive Summary

Business Problem

Space Y must determine launch pricing to compete with SpaceX. Key cost driver: whether Falcon 9's first stage is recovered and reused (~\$62M vs \$165M+).

\$62M

SpaceX Falcon 9

Our Approach

Collect SpaceX launch data via API & web scraping, perform EDA with SQL & visualization, build interactive maps & dashboards, then train ML classifiers.

>\$165M

Competitors

Key Outcome

A trained classification model predicts first-stage landing success, enabling accurate cost estimation for each mission.

~30%

Cost Saved by Reuse

Introduction

- The commercial space industry is growing rapidly, with companies like Virgin Galactic, Rocket Lab, Blue Origin, and SpaceX making launches more accessible.
- SpaceX stands out with landmark achievements:
 - Resupply missions to the International Space Station
 - Starlink satellite internet constellation
 - Crewed orbital missions
- The Falcon 9's advertised launch cost is \$62M — far below competitors at \$165M+ — thanks to first-stage reuse.
- If the first stage lands successfully, costs drop dramatically. Predicting this outcome is the core business question.

Project Context

Role:	Data Scientist at Space Y
Goal:	Predict first-stage landing success
Data Source:	SpaceX REST API + Web Scraping
Tools:	Python, SQL, Folium, Plotly Dash, Scikit-learn
Output:	Classification model + interactive dashboards

Data Collection & Data Wrangling Methodology

01

SpaceX REST API

GET requests to `api.spacexdata.com/v4/launches/past`. JSON responses normalized via `json_normalize()` into a flat DataFrame.

02

Web Scraping

BeautifulSoup used to scrape Falcon 9 launch records from Wikipedia HTML tables, parsed into Pandas DataFrames.

03

ID Resolution

Columns like 'rocket' contain IDs only. Additional API calls to `/booster`, `/launchpad`, `/payload`, `/core` endpoints resolve them to readable values.

04

Sampling

Falcon 1 launches filtered out; only Falcon 9 records retained for analysis.

05

Null Handling

PayloadMass nulls replaced with column mean. LandingPad nulls kept (no pad = no landing) for one-hot encoding later.

06

Class Labeling

Outcome column converted to binary class Y: 1 = successful landing, 0 = failed landing — the target variable.

EDA & Interactive Visual Analytics Methodology

EDA with Visualization

- Scatter plots of payload vs. launch outcome
- Bar charts by launch site and orbit
- Line charts of success rate over time
- Heatmap of feature correlations
- One-hot encoding of categorical variables

Folium Interactive Maps

- Mark launch sites on a world map
- Color-coded outcome markers
- Proximity analysis (coastline, cities, highways)
- Identify geographic patterns for site selection

EDA with SQL

- SQLite queries on launch data
- Aggregate success rates by site and orbit
- Filter and group to uncover patterns
- Foundation for ML feature engineering

Plotly Dash Dashboard

- Dropdown to filter by launch site
- Range slider for payload mass
- Dynamic pie chart: success/failure split
- Scatter chart: payload vs. outcome
- Real-time filter-linked visualizations

Predictive Analysis Methodology



Algorithms Evaluated

Logistic Regression

Baseline binary classifier using sigmoid function. Effective for linearly separable features.

Support Vector Machine

Finds optimal hyperplane maximizing margin between classes. Handles high-dimensional data well.

Decision Tree

Hierarchical rule-based splits. Interpretable and handles non-linear boundaries.

K-Nearest Neighbors

Classifies based on majority vote among K closest training samples in feature space.

Best hyperparameters found via GridSearchCV. Final model selected by highest accuracy on training data.

Data Collection — SpaceX API

1 Import Libraries

requests, pandas, numpy, datetime
— core tools for HTTP calls & data handling.

2 GET /v4/launches/past

requests.get(spacex_url) → HTTP 200. Parses JSON with pd.json_normalize() into flat DataFrame.

3 Subset & Filter

Keep: rocket, payloads, launchpad, cores, flight_number, date_utc. Remove multi-core / multi-payload rows. Filter dates ≤ 2020-11-13.

4 ID Resolution (4 API calls)

getBoosterVersion() · getLaunchSite() · getPayloadData() · getCoreData() — each hits a child endpoint to resolve IDs → readable values.

5 Build launch_dict

16 columns assembled into a Python dict: FlightNumber, Date, BoosterVersion, PayloadMass, Orbit, LaunchSite, Outcome, Flights, GridFins, Reused, Legs, LandingPad, Block, ReusedCount, Serial, Longitude, Latitude.

6 Filter Falcon 9 Only

df[df['BoosterVersion'] != 'Falcon 1'] → data_falcon9. FlightNumber reset to sequential index.

7 Handle Nulls

PayloadMass: fillna(mean). LandingPad: retain None (no pad used). Confirm isnull().sum() = 0 except LandingPad.

8 Export dataset_part_1.csv

Final clean DataFrame exported. 90 Falcon 9 records, 17 columns, ready for wrangling.

```
response = requests.get('https://api.spacexdata.com/v4/launches/past')
data = pd.json_normalize(response.json()) # → 200 OK, 92 rows × 31 cols
```

**90 Falcon 9 records
17 features extracted**

Notebook Reference:

github.com/Temiarubi/spacex-project · [jupyter-labs-spacex-data-collection-api.ipynb](#)

Data Collection — Web Scraping (Wikipedia)

1 Target URL

Wikipedia snapshot (9 Jun 2021):
List of Falcon 9 and Falcon Heavy launches ·
oldid=1027686922

2 HTTP GET + BeautifulSoup

```
requests.Session().get(static_url) → soup =  
BeautifulSoup(response.text, 'html.parser')
```

3 Locate Launch Tables

```
soup.find_all('table', 'wikitable plainrowheaders  
collapsible') → 3 HTML tables containing launch  
records.
```

4 Extract Column Headers

Iterate <th> tags via `extract_column_from_header()`
→ 10 column names: Flight No., Date, Version
Booster, Launch Site, Payload, Payload mass, Orbit,
Customer, Launch outcome, Booster landing.

5 Parse Table Rows

For each <tr> where `th.string.isdigit()`: extract
`date_time()`, `booster_version()`, `get_mass()`,
`landing_status()` using helper functions.

6 Build DataFrame

```
defaultdict(list) → launch_dict →  
pd.DataFrame(launch_dict). Handles Wikipedia  
noise: refs [b], N/A, inconsistent formatting.
```

```
soup = BeautifulSoup(response.text, 'html.parser')  
tables = soup.find_all('table', 'wikitable plainrowheaders collapsible')
```

**3 HTML tables parsed
Falcon 9 launch records extracted**

Notebook Reference:

github.com/Temiarubi/spacex-project · [jupyter-labs-webscraping.ipynb](#)

Data Wrangling

1 Inspect Nulls

`df.isnull().sum()/len(df)*100` → PayloadMass has nulls; LandingPad intentionally blank.

2 Count by LaunchSite

`df['LaunchSite'].value_counts()` → CCAFS SLC 40, VAFB SLC 4E, KSC LC 39A confirmed as 3 active sites.

3 Count by Orbit

`df['Orbit'].value_counts()` → GTO (27), ISS (21), VLEO (14) are top 3 orbit types.

4 Outcome Classification

`landing_outcomes = df['Outcome'].value_counts()`
→ 8 unique outcome strings (True/False × ASDS/RTLS/Ocean/None).

5 Create Class Column

`bad_outcomes = {False ASDS, False Ocean, False RTLS, None ASDS, None None}`
Class = 0 if bad_outcome else 1

6 Export & Verify

`df['Class'].mean()` confirms landing success rate.
Export to `dataset_part_2.csv` for EDA.

Landing Type

Class

True ASDS / True RTLS / True Ocean

1 ✓ (Success)

False ASDS / False RTLS / False Ocean / None ASDS / None None

0 ✗ (Failure)

dataset_part_2.csv
~80 Falcon 9 records
Class: 0 or 1

Notebook Reference:

github.com/Temiarubi/spacex-project · `labs-jupyter-spacex-Data_wrangling.ipynb`

EDA with Visualization — Results

T1 Flight No. vs Launch Site  Scatter Why: Reveals whether newer flights (higher numbers) favour specific launch sites, and how success rate shifts across sites over time. ▶ Later flights show higher Class=1 rate; KSC LC-39A dominates recent missions.	T2 Payload Mass vs Launch Site  Scatter Why: Identifies whether heavy payloads are preferentially launched from certain sites, and if mass limits exist per site. ▶ VAFB SLC-4E has no launches above 10,000 kg — a hard operational constraint.	T3 Success Rate by Orbit  Bar Chart Why: Bar charts aggregate the binary Class variable per orbit, making success probability directly comparable across orbit types. ▶ ES-L1, HEO, and SSO hit 100% success; GTO remains the most challenging orbit.
T4 Flight Number vs Orbit  Scatter Why: Checks whether SpaceX's organisational learning (captured by flight number) differentially improves success across orbits. ▶ LEO success correlates with flight count; GTO shows no such trend — orbit energy matters more.	T5 Payload Mass vs Orbit  Scatter Why: Tests whether mass-to-orbit constraints drive landing outcomes; heavier payloads leave less fuel for recovery burns. ▶ Polar, LEO, ISS succeed even with heavy loads; GTO outcomes remain mixed regardless of mass.	T6 Yearly Success Trend  Line Chart Why: A line chart over time surfaces the cumulative effect of engineering improvements, crew experience, and booster reuse maturity. ▶ Success rate rose steadily from ~20% (2013) to ~80%+ (2020), validating year as a key ML feature.

Notebook Reference:

github.com/Temiarubi/spacex-project · [edadataviz.ipynb](#)

EDA with SQL — Results

T1 <pre>SELECT DISTINCT Launch_Site FROM SPACEXTABLE;</pre> ▶ 3 unique sites: CCAFS SLC 40, VAFB SLC 4E, KSC LC 39A	T2 <pre>SELECT * FROM SPACEXTABLE WHERE Launch_Site LIKE 'CCA%' LIMIT 5;</pre> ▶ 5 CCAFS records confirmed with date, payload & outcome	T3 <pre>SELECT SUM(PAYLOAD_MASS_KG_) ... WHERE Customer = 'NASA (CRS)';</pre> ▶ Total NASA CRS payload mass aggregated across all missions	T4 <pre>SELECT AVG(PAYLOAD_MASS_KG_) ... WHERE Booster_Version = 'F9 v1.1';</pre> ▶ Average payload mass for F9 v1.1 booster version computed	T5 <pre>SELECT Date ... WHERE Landing_Outcome = 'Success (ground pad)' LIMIT 1;</pre> ▶ First ever ground-pad success date identified
T6 <pre>SELECT DISTINCT Booster_Version WHERE drone ship AND 4000 < mass < 6000;</pre> ▶ Boosters with drone-ship success in mid-mass range listed	T7 <pre>SELECT Mission_Outcome, COUNT(*) ... GROUP BY Mission_Outcome;</pre> ▶ Success vs. failure mission count breakdown by outcome label	T8 <pre>SELECT Booster_Version WHERE PAYLOAD_MASS_KG_ = (SELECT MAX...);</pre> ▶ Booster(s) carrying the heaviest payload identified via subquery	T9 <pre>SELECT month, Booster_Version, Launch_Site WHERE year='2015' AND Failure AND drone ship;</pre> ▶ 2015 drone-ship failures: Jan & Apr, F9 v1.1, CCAFS SLC 40	T10 <pre>SELECT Landing_Outcome, COUNT(*) ... 2010-2017 GROUP BY ... ORDER BY DESC;</pre> ▶ Success (ground pad) ranked 1st in 2010–2017 period outcomes

Key SQL Findings

- ▶ KSC LC-39A & VAFB SLC-4E are the most productive sites by success rate
- ▶ Drone-ship landing failures clustered in 2015 (Jan & Apr) for F9 v1.1 at CCAFS
- ▶ Success (ground pad) was the most frequent positive outcome 2010–2017
- ▶ Heaviest payload boosters identified for mission cost analysis

Notebook Reference:

github.com/Temiarubi/spacex-project · [jupyter-labs-eda-sql-coursera_sqllite.ipynb](#)

Interactive Map Results — Folium

TASK 1

Mark All Launch Sites

- ▶ folium.Map() centred at NASA JSC (Houston)
- ▶ folium.Circle() + folium.Marker() per site
- ▶ 4 sites plotted: CCAFS LC-40, CCAFS SLC-40, KSC LC-39A, VAFB SLC-4E
- ▶ All sites within 20 km of coastline confirmed

TASK 2

Success / Failure Markers

- ▶ MarkerCluster() to group overlapping points
- ▶ Class=1 → green marker, Class=0 → red marker
- ▶ `spacex_df['marker_color'] = class.map({1:'green', 0:'red'})`
- ▶ KSC LC-39A visually shows highest green density

TASK 3

Proximity Distances

- ▶ MousePosition plugin to record coordinates
- ▶ Haversine formula:
`calculate_distance(lat1,lon1,lat2,lon2)`
- ▶ Measured: coastline, highway, railway, nearest city
- ▶ KSC coastline distance ≈ 0.56 km — closest of all sites

```
distance = calculate_distance(28.56367, -80.57163, # KSC
                             28.56320, -80.56700) # coastline
# → 0.56 km | PolyLine drawn on map
```

Coastal Proximity

All 4 launch sites sit within 20 km of the Atlantic/Pacific coast — rocket trajectories fly over ocean for safety.

Equatorial Advantage

CCAFS sites (28.5°N) are closest to the equator, maximising orbital insertion efficiency for GTO missions.

Optimal Site

KSC LC-39A: highest success rate + closest coastline + best equatorial position = top-ranked launch site.

Notebook Reference:

github.com/Temiarubi/spacex-project · `lab_jupyter_launch_site_location.ipynb`

Plotly Dash Dashboard — Results

App Architecture

Data Source:

dataset_part_2.csv (SpaceX launches)

Dropdown:

site-dropdown → ALL + 3 individual sites (CCAFS SLC 40, KSC LC 39A, VAFB SLC 4E)

Range Slider:

payload-slider: 0–10,000 kg | min=350, max=15,600 kg in data

Pie Chart:

ALL: success count per site (KSC 55%, CCAFS 28.3%, VAFB 16.7%)

Single site: Success vs Failure split

Scatter Chart:

PayloadMass vs Class (0/1), colour-coded by BoosterVersion (Falcon 9)

Callbacks:

@app.callback links dropdown + slider → pie chart

@app.callback links dropdown + slider → scatter chart

Dashboard Insights

KSC LC-39A: 55% of all successes

Dominates the All Sites pie chart — highest success volume across the dataset.

Payload sweet spot: ~1,000–5,500 kg

Scatter chart shows Class=1 clusters densely in this mass band across all sites.

Falcon 9 only booster in dataset

BoosterVersion colour in scatter = single legend entry (Falcon 9) — clean dataset.

High-mass failures > 6,000 kg

Class=0 points appear more frequently above 6,000 kg, especially at CCAFS SLC 40.

Real-time interactivity confirmed

Dropdown + slider callbacks update both charts simultaneously; no page reload.

All Sites payload range: 350–15,600 kg

Slider min/max derived from actual data; confirms broad mission mass diversity.

Notebook Reference:

github.com/Temiarubi/spacex-project · [spacex-dash-app.py](#)

Predictive Analysis — Classification Results

Logistic Regression

Best params:
C=0.01, penalty=l2, solver=lbfgs

CV Accuracy Test Accuracy
~84% **83.3%**

Support Vector Machine

Best params:
kernel=linear, C=0.1 (or rbf+gamma)

CV Accuracy Test Accuracy
~84% **83.3%**

Decision Tree

Best params:
GridSearchCV 10-fold tuned

CV Accuracy Test Accuracy
86.25% **88.9%**

K-Nearest Neighbors

Best params:
n_neighbors, algorithm, p tuned

CV Accuracy Test Accuracy
~84% **83.3%**

StandardScaler
(preprocessing)

train_test_split
test_size=0.2, rs=2

GridSearchCV
cv=10, scoring=accuracy

Best Model
Decision Tree

Confusion Matrix
Visualized

Confusion Matrix — Decision Tree (Best Model)

	Pred: Land	Pred: No Land
Act: Land	16 ✓	2 ✗
Act: No Land	1 ✗	5 ✓

Decision Tree — Best Model: 88.9% test accuracy

Trained with 10-fold GridSearchCV. Outperforms all other classifiers on held-out test set.

All models tied at 83.3% except Decision Tree

LR, SVM, KNN all reach the same 83.3% ceiling — Decision Tree's non-linear splits capture interactions the others miss.

False positives are the primary error type

Confusion matrix shows 2 false positives (predicted land, actually didn't) vs. 1 false negative — cost asymmetry important for mission planning.

Notebook Reference:

github.com/Temiarubi/spacex-project · [SpaceX_Machine_Learning_Prediction_Part_5.ipynb](#)

Conclusion

Data Collection

SpaceX REST API + Wikipedia web scraping (BeautifulSoup) yielded 90 cleaned Falcon 9 records with 17 features across 3 active launch sites.

EDA & SQL

10 SQL queries confirmed: KSC LC-39A leads in success volume (55%), drone-ship failures clustered in 2015, ES-L1/HEO/SSO orbits achieved 100% success rate.

Interactive Analytics

Folium maps confirmed all sites are coastal (KSC $\approx$ 0.56 km from coast). Plotly Dash revealed payload sweet spot: 1,000–5,500 kg maximises Class=1 probability.

ML Classification

Decision Tree Classifier achieved the best performance: 86.25% CV accuracy, 88.9% test accuracy — outperforming LR, SVM, and KNN (all at 83.3%).

Space Y can now predict first-stage landing outcomes with 88.9% accuracy — enabling data-driven launch cost estimation (\$62M vs \$165M+) to compete directly with SpaceX.

Innovative Insights

Geography IS a Feature — All Sites are Coastal

Folium proximity analysis showed KSC is only 0.56 km from the coast. Coastal positioning enables both ocean-trajectory safety and ASDS drone-ship recovery. Inland sites cannot land boosters via RTLS without range violations — geography implicitly encodes mission parameters.

Payload Mass 1,000–5,500 kg = Landing Sweet Spot

EDA scatter charts (T2, T5) and Dash interactivity together confirm Class=1 dominates this band across all 3 sites. Above 6,000 kg, return-burn propellant budget is exhausted — especially for GTO trajectories. This creates a non-linear decision boundary linear models miss (hence Decision Tree outperforms LR/SVM).

2015 Was the Critical Pivot Year

SQL Task 9 surfaced the 2 drone-ship failures in Jan & Apr 2015 (F9 v1.1, CCAFS). The line chart (T6 EDA) shows the success rate inflected sharply upward post-2015 as Block 3/4/5 boosters replaced v1.1. Year acts as a latent composite variable encoding booster maturity, software updates, and crew experience simultaneously.

Decision Tree Outperforms Because Rocket Success is Non-Linear

LR and SVM (both linear) plateau at 83.3%. The Decision Tree's hierarchical splits natively capture the interaction effects revealed in EDA: site × payload × orbit × year all combine non-linearly to determine landing success. The 5.6% accuracy gap over linear models validates the EDA finding that no single feature dominates.

All insights grounded in notebook findings: EDA (T1–T6), SQL (T1–T10), Folium (T1–T3), Dash app, and ML (Decision Tree 88.9%).

GitHub Repository

spacex-project

Temiarubi · GitHub

github.com/Temiarubi/spacex-project

Jupyter Notebooks

Data Collection via SpaceX API

Web Scraping with BeautifulSoup

EDA with Visualization

EDA with SQL

Interactive Map (Folium)

Predictive Analysis (ML)

Python Files

Plotly Dash Dashboard app.py

Data wrangling utility scripts

visit the URL above to access all completed notebooks and source files.